

Understanding Auto-Scheduling Optimizations for Model Deployment via Visualizations

Laixin Xie¹, Chenyang Zhang², Ruofei Ma¹, Xing Jiang³,
Xingxing Xing³, Wei Wan³, Quan Li¹

1. School of Information Science and Technology,
ShanghaiTech University
2. University of Illinois at Urbana-Champaign
3. UX Center, Netease Games



上海科技大学
ShanghaiTech University



ViSeer LAB

交互智能与
可视分析实验室

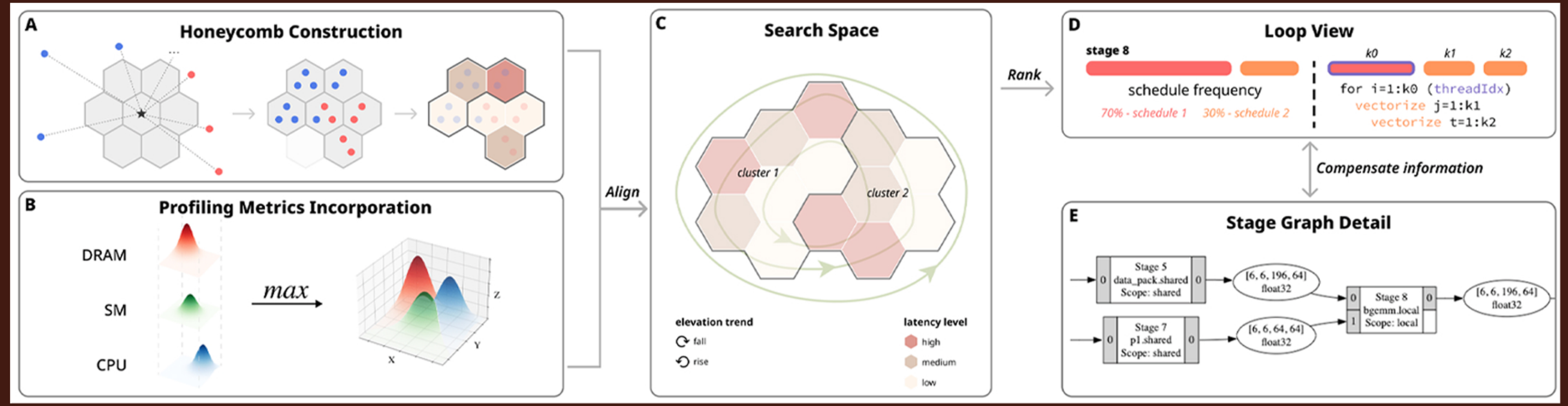


Figure 1: (A) Honeycomb Chart Construction; (B) Hardware Performance Comparison; (C) Honeycomb Chart and Contour Map in Search Space Visualization; (D) Example Entry in the Loop View; (E) Stage Graph visualization.

ABSTRACT

After completing the design and training phases, deploying a deep learning model onto specific hardware is essential before practical implementation. Targeted optimizations are necessary to enhance the model's performance by reducing inference latency. Autoscheduling, an automated technique offering various optimization options, proves to be a viable solution for large-scale auto-deployment. However, the low-level code generated by auto-scheduling resembles hardware coding, potentially hindering human comprehension and impeding manual optimization efforts. In this ongoing study, we aim to develop an enhanced visualization that effectively addresses the extensive profiling metrics associated with auto-scheduling. This visualization will illuminate the intricate scheduling process, enabling further advancements in latency optimization through insights derived from the schedule.

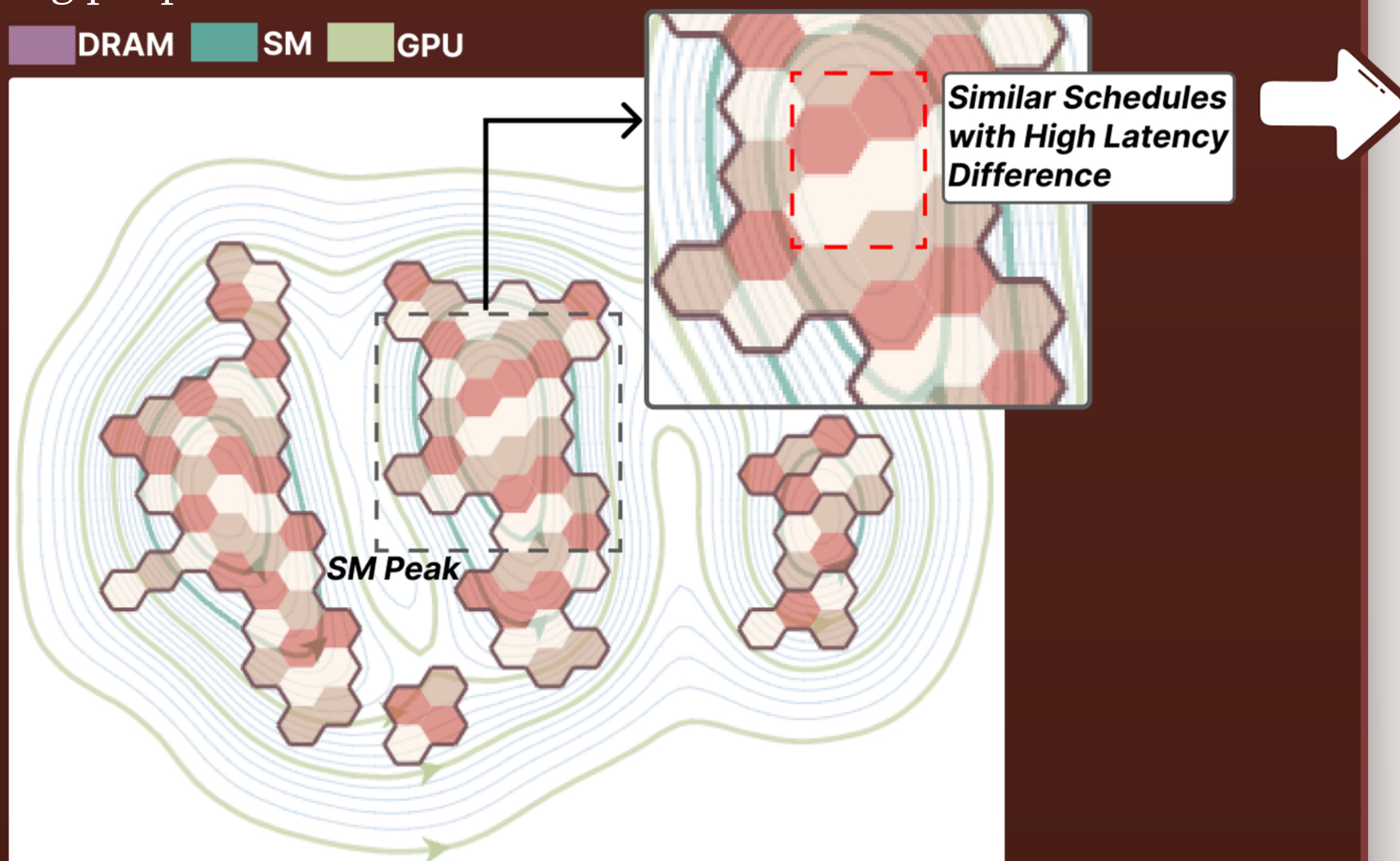
2 VISUAL DESIGNS

We propose two visual designs for hierarchical analysis of autoscheduling results. The first design (Fig.1 (C)) focuses on exploring the search space documented in Ansor's searching log, which captures the search process over a computational graph with coarse operators like convolution, encompassing operations such as addition and multiplication. The second design, known as the "loop view" (Fig.1 (D)), presents a visually intuitive schedule where each schedule corresponds to a fine-grained operation like BGEMM (Batched General Matrix Multiply), typically involving a single hardware instruction known as the "stage graph". Our visualizations break down the task graph into subtasks and transform them into stages, facilitating meticulous optimization of the computational graph.

2.1 Search Space Visualization

Constructing the Optimization Search Space. Employing dimension reduction and clustering techniques, we analyze and compare similarities among the optimization options. To visualize these options in a two-dimensional space while preserving their similarity, we introduce the "honeycomb chart". The chart is constructed step-by-step, with cells formed outward from the clustering center, prioritizing points closer to the core (Fig.1 (A)). Further partitioning of the honeycomb enhances understanding by assigning each cell a corresponding category based on the majority category within it. Boundaries are established between cells of different categories. Cell color encodes the average inference latency for optimized options.

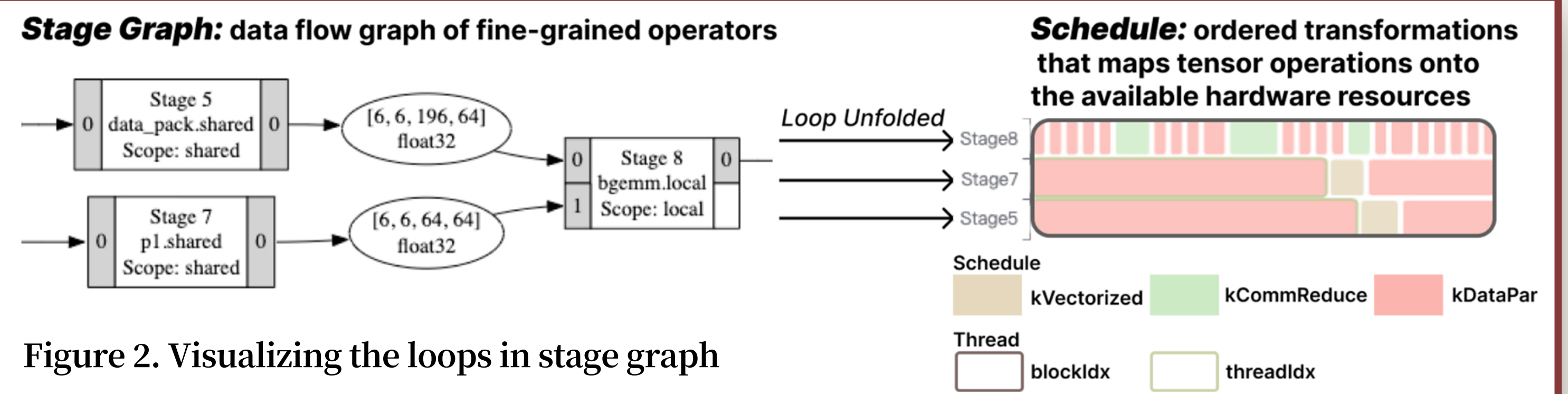
Incorporating Hardware Profiling Metrics. Profiling metrics are categorized into three groups based on the corresponding hardware components: Dynamic Random Access Memory (DRAM), Streaming Multiprocessor (SM), and Graphics Processing Unit (GPU). We merge the distinct peaks of performance improvement by selecting the maximum value among them as the Z-axis value for each point in the XY plane (Fig. 1(B)). This merging process indicates the hardware type with the most substantial performance improvement due to associated optimization options within the honeycomb space. The resulting three-dimensional peaks are transformed into contour lines within the two-dimensional honeycomb space. The below figure demonstrates this visualization used to identify the critical schedules for learning purpose.



2.2 Understanding Optimization Option

The "Loop View" consists of two components to explore scheduling strategies within each stage of the fine-grained computational graph. The left section shows the distribution of scheduling strategies based on frequency, while the right section reveals the proportion of loops with distinct scheduling strategies. For example, in "stage 8", two scheduling strategies are employed, with 70% using type 1 and 30% using type 2 (Fig. 1(D)). The number of loops with each strategy is indicated. The "Stage Graph Detail" (Fig. 1(E)) dissects data flow between different stages within the selected task, providing users with rich contextual information for evaluating optimization strategies for each stage in the scheduling. Fig. 2 illustrates the connection between these two views.

3 EVALUATION



In a case study with a domain expert, we utilized the ResNet-18 model. Through our visual designs, the expert explored and analyzed the optimization options, leading to the derivation of two design principles for the parameter of two parallelism techniques: commutative reducer and vectorization.

1) Shorten the commutative reducer in stage 8: The expert observed reduced latency in the BGEMM operation with a smaller commutative reducer (OP1-4 comparison in Fig.3). Based on this, the expert proposed design principle 1 (Fig.3 P1), suggesting incremental iteration of the final commutative reducer's search range within BGEMM's loop.

2) Shorten the vectorization in stage 5: Comparing OPT3 with OPT4 (Fig.3), the expert identified the potential to reduce latency by shortening the vectorization in stage 5. A shorter vectorization call increases throughput, particularly beneficial for stage 5 with numerous small loops. Thus, design principle 2 (Fig.3 P2) suggests exploring a narrow range of values to determine the optimal length of the vectorization operation in stage 5.

In conclusion, the expert effectively derived two design principles using our visual designs, enabling quick identification of optimal values and improved performance. This outcome provides strong evidence of the effectiveness of our visual designs in aiding the expert's analysis and decision-making process.

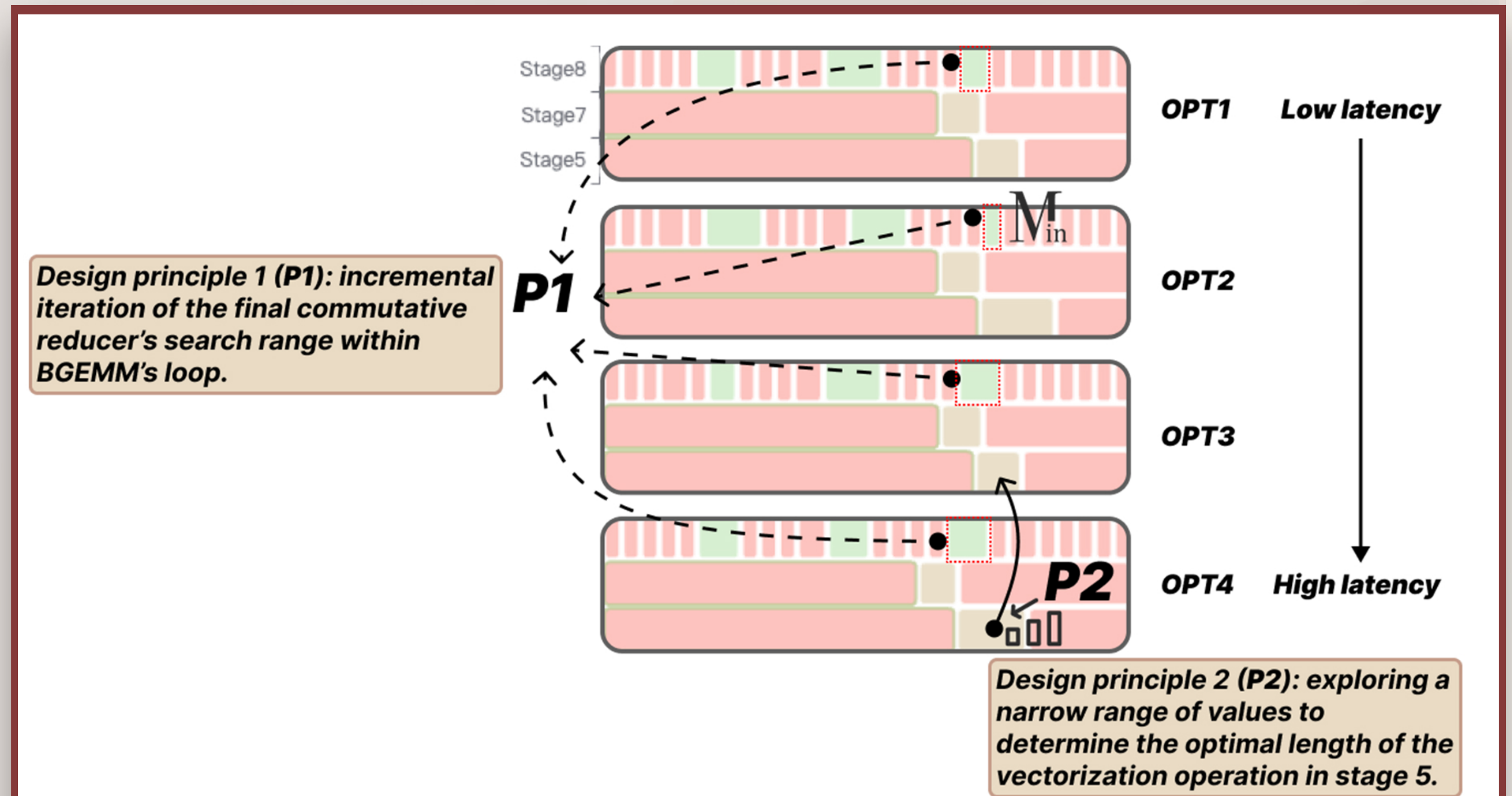


Figure 3: Four representative schedules extracted from the search space visualization. These schedules have enabled an expert to formulate two key design principles.

4 DISCUSSION AND FUTURE WORK

The deployment of deep learning models is vital for their widespread adoption. As Large Language Models (LLM) progress, achieving low inference latency becomes crucial for enhancing user experience by reducing waiting times for answers. Our contribution lies in the field of visualization for model deployment techniques. Our goal is to develop an analytical workflow and a visual analytics system with user feedback. We aim to evaluate the system's effectiveness in supporting deployers and reducing inference latency using quantitative and qualitative methods.